COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE12S4
DATE PERFORMED:	07/24/26	**DATE SUBMITTED:**	
NAME:	Francis Elijah J. Formalejo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Main.cpp

```cpp
#include <iostream>
#include "doubly_ll.h"

int main() {
    DoubleList<char>* head = CreateNewNode('C');
    DoubleList<char>* second = CreateNewNode('P');
    DoubleList<char>* third = CreateNewNode('E');

    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    std::cout << "Testing the DLL traversal" << std::endl;
    dllTraverse(head);

    std::cout << "Testing the insertion at the head node" << std::endl;
    dllInsertHead('X', &head);
    dllTraverse(head);

    std::cout << "Testing insertion at the end node" << std::endl;
    dllInsertEnd('0', head);
    dllTraverse(head);

    std::cout << "Testing general insertion at position 2" << std::endl;
    dllInsertGen('Z', head, 2);
    dllTraverse(head);

    return 0;
}
```

Doubly:

```cpp
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H
#include <iostream>

template <typename T>
class DoubleList {
public:
    T data;
    DoubleList<T>* prev = nullptr;
    DoubleList<T>* next = nullptr;
};

template <typename T>
DoubleList<T>* CreateNewNode(T newData) {
    DoubleList<T>* newNode = new DoubleList<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

template <typename T>
void dllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    if (currentNode == nullptr) {
        std::cout << "The list is empty";
        return;
    }

    std::cout << "Forwarding: ";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << std::endl;

    std::cout << "Backwarding: ";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}

template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead) {
```

```cpp
void dllInsertHead(T newData, DoubleList<T>** currentHead) {
    DoubleList<T>* newNode = CreateNewNode(newData);
    newNode->next = *currentHead;

    if (*currentHead != nullptr) {
        (*currentHead)->prev = newNode;
    }

    *currentHead = newNode;
}

template <typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead) {
    if (currentHead == nullptr) return;

    DoubleList<T>* newNode = CreateNewNode(newData);

    while (currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }

    currentHead->next = newNode;
    newNode->prev = currentHead;
}

template <typename T>
void dllInsertGen(T newData, DoubleList<T>* currentHead, int position) {
    if (currentHead == nullptr) return;

    DoubleList<T>* newNode = CreateNewNode(newData);

    for (int i = 1; i < position - 1; i++) {
        if (currentHead->next != nullptr) {
            currentHead = currentHead->next;
        }
    }

    newNode->next = currentHead->next;
    newNode->prev = currentHead;

    if (currentHead->next != nullptr) {
        currentHead->next->prev = newNode;
    }
    currentHead->next = newNode;
}

#endif
```

```
Testing the DLL traversal
Forwarding: C P E
Backwarding: E P C
Forwarding: C P E
Backwarding: E P C
Backwarding: E P C
Testing the insertion at the head node
Testing the insertion at the head node
Forwarding: X C P E
Backwarding: E P C X
Forwarding: X C P E
Backwarding: E P C X
Testing insertion at the end node
Testing insertion at the end node
Forwarding: X C P E 0
Forwarding: X C P E 0
Backwarding: 0 E P C X
Testing general insertion at position 2
Backwarding: 0 E P C X
Testing general insertion at position 2
Testing general insertion at position 2
Forwarding: X Z C P E 0
Forwarding: X Z C P E 0
Backwarding: 0 E P C Z X
Backwarding: 0 E P C Z X
```

I used a double pointer for the head insert so the main pointer updates okay and my traversal function captures the tail while the forward pass to check the backward linkings. Then the driver code checks the insertion cases and based on the output, the forward and backward are in bluetooth (they have connection lol).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:
Overall, I enjoyed this activity even tho it took me a hard time to understand and catch up but eventually, I got into snowball phase, where I can understand it right then, I was also shocked. I also learned how dynamic data structures work like singly and doubly for two way traversals. Its like the pointers are the headquarters and it commands the head and tail with forwarding and backwarding traversals. 7/10 difficulty btw.